

Desarrollo De Una TDA Con Un Entero

Estructura de Datos

Reporte

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Horario: Lun, Mie y Vie de 3:00 a 5:00 P.M.

Profesor: Noé Ortega Sánchez

Fecha: 07/09/2025

Contenido

Introducción al Problema.....	3
Definición de una lista.....	3
Diagrama de clases.	4
Diccionario de clases.....	5
Conclusión.....	7

Introducción al Problema.

Se requiere hacer una lista dinámica la cuál registre distintos valores tipo entero, una donde nos permita realizar múltiples operaciones como insertar, buscar y eliminar. Al ser dinámica no podemos apoyarnos con el uso de vectores, pues para usarlos, se necesita definir su tamaño en compilación y nosotros desconocemos el tamaño de los datos que se almacenarán en esa lista.

Es por eso por lo que vamos a utilizar un tipo de dato abstracto (TDA). En concreto, una lista simplemente ligada que nos va a resultar bastante útil para la resolución que buscamos a este problema.

Definición de una lista.

Una lista simplemente ligada está conformada por dos clases: la clase Nodo y la clase Lista. Un nodo se refiere a la celda donde se está guardando la información a almacenar, ya sea un entero, flotante, carácter o incluso otra clase (en nuestro caso es un entero el que se almacenará) y la dirección donde se encuentra el nodo siguiente en la lista. En la clase Nodo se definen el tipo de variable que se introducirá en la lista y un puntero de su misma clase Nodo que guardar la dirección del siguiente nodo. Mientras que, dentro de la clase Lista, se definirán todas las operaciones que se podrán realizar con la lista y un puntero al primer nodo que se vaya a crear, el nodo cabecera. Además, es preciso crear una interfaz de menú para la creación de la lista y poder acceder a sus diferentes tipos de operaciones.

Diagrama de clases.

Con esto planteado, vamos a revisar el diccionario de clases y el diagrama de interacción entre estas que, se necesitaron para resolver el problema planteado.

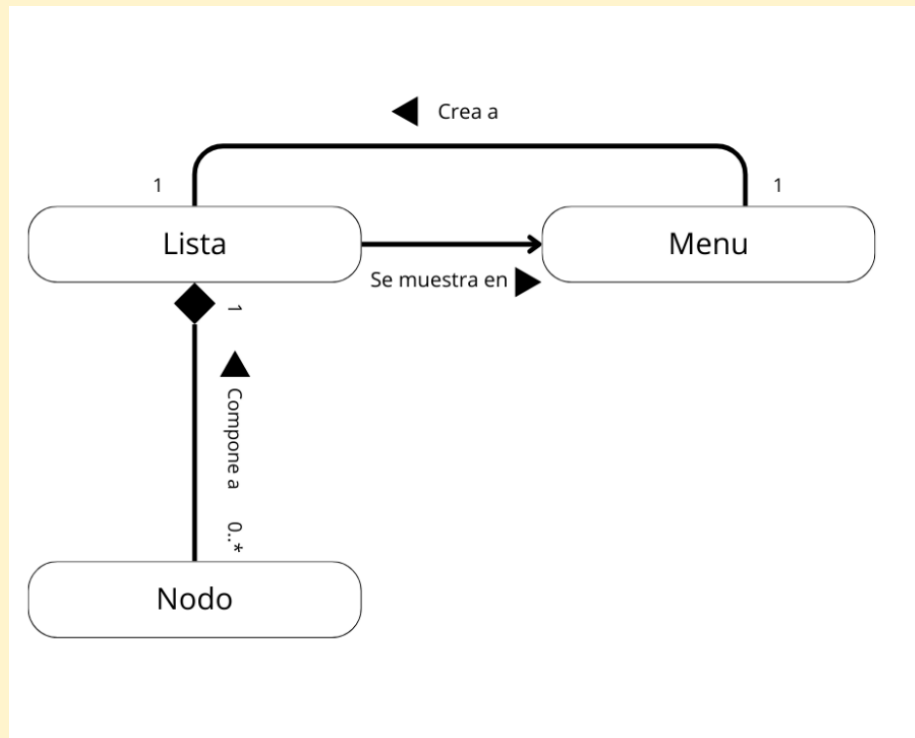


Ilustración 1: Diagrama de clases

Las relaciones entre estas son muy sencillas, Menu es la clase con la que interactúa el usuario y solamente esta es capaz de acceder a la creación de la lista y realizar las operaciones que se requieran. La clase lista se compone de distintos nodos que esta misma va creando conforme se van necesitando, ya sea añadiendo elementos o eliminando elementos de la lista.

Diccionario de clases

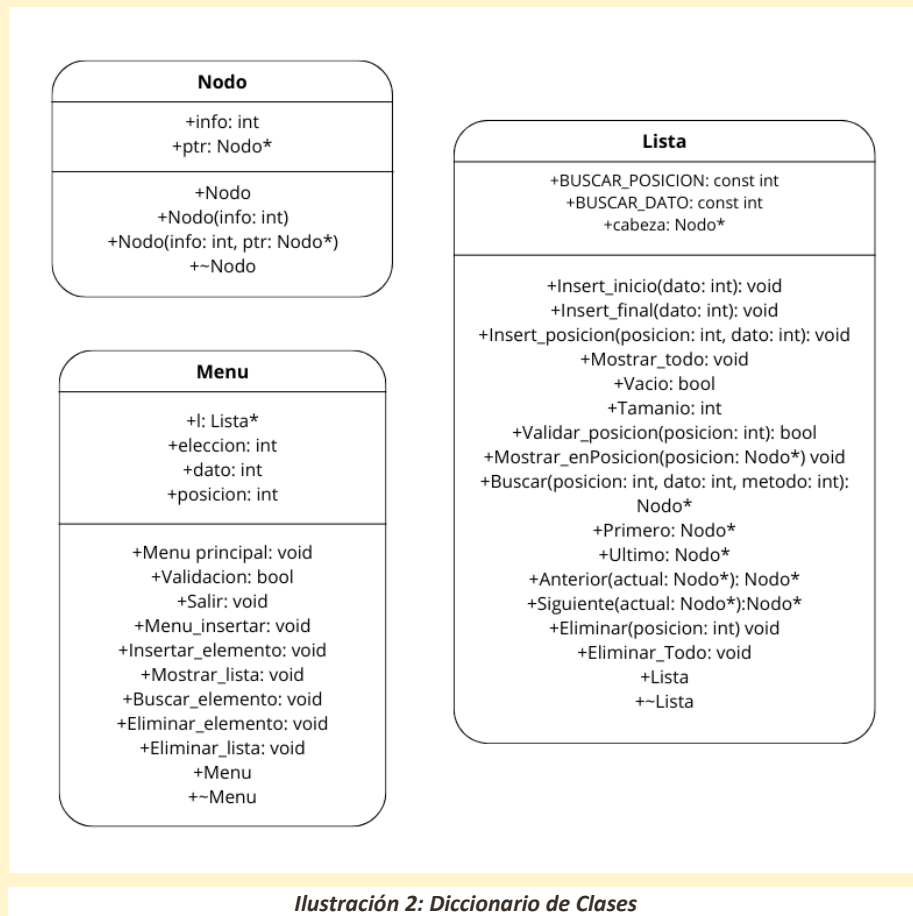


Ilustración 2: Diccionario de Clases

El comportamiento del menú es:

1. **Menu_principal:** Accede al menú principal, donde se encuentran las principales acciones de la lista, como: Inicializar la lista, insertar un elemento, mostrar la lista, buscar un elemento dentro de la lista y eliminar la lista.
2. **Validacion:** Verifica que ya esté creada una lista antes de empezar a operar con ella. Es vital para no acceder a espacios de memoria ilegales.
3. **Salir:** Sale del programa.
4. **Inicializar_lista:** Crea la lista y devuelve si la lista fue creada exitosamente o hubo un problema al crearla.
5. **Insertar_elemento:** Despliega el menú de opciones de insertar: insertar al inicio, insertar al final, insertar en una posición indicada.
6. **Mostrar_lista:** Muestra los elementos existentes de la lista.
7. **Buscar_elemento:** Despliega el menú de búsqueda, ya sea si quieres buscar un elemento en la posición o por la información.
8. **Eliminar_elemento:** Elimina un elemento deseado de la lista.

9. *Eliminar_lista*: Se encarga de eliminar los elementos de toda la lista y la lista en sí, necesitas crear una nueva lista para poder operar con ella de nuevo.
10. *Menu*: Crea la clase Menú.
11. *~Menu*: Borra el espacio utilizado por Menu.

El comportamiento de la lista es la siguiente:

1. *Insert_inicio*: Crea un nuevo nodo, y el dato que tiene guardado lo manda a la información del nodo y se le asigna la dirección del nodo que tenía anteriormente la cabeza, y la dirección de la cabeza pasa a ser la del nuevo nodo.
2. *Insert_final*: Recorre toda la lista hasta encontrar el último nodo, una vez obtenido, crea un nuevo nodo, le sede la información y al ultimo anterior le asigna la dirección de este nuevo, mientras que el nuevo apunta ahora a nulo.
3. *Insert_posicion*: Crea un nodo y lo inserta donde tu desees, al insertar en una posición seleccionada desplaza ese nodo hacia adelante y le da el lugar al nuevo, en caso de seleccionar una posición mayor a la existente el nodo simplemente inserta el nodo al final.
4. *Mostrar_todo*: Muestra todos los nodos, empieza desde la cabecera hasta el nodo que apunta a nulo.
5. *Vacio*: Determina si la lista se encuentra vacía.
6. *Tamano*: Te da el tamaño de la lista como un número entero natural.
7. *Validar_posicion*: Registra que la posición dada sea un número entero natural.
8. *Mostrar_enPosicion*: Recibe la dirección de un nodo y checa si es una dirección valida. Evita accesos de memoria ilegales.
9. *Buscar*: Recibe un dato o una dirección, y el método por el cual se va a realizar la búsqueda. Si el método es dato, checa toda la lista hasta encontrar la primera incidencia, sino entonces te avisa que no ha encontrado nada. Igual con la posición, busca hasta encontrar la posición señalada, sino te avisa que no encontró nodo.
10. *Primero*: Devuelve la dirección del primer nodo en la lista.
11. *Ultimo*: Devuelve la dirección del último nodo en la lista.
12. *Anterior*: Dada una dirección, devuelve la dirección del nodo anterior al señalado.
13. *Siguiente*: Dada una dirección, devuelve la dirección del nodo siguiente al señalado.
14. *Eliminar*: Dada una posición, busca el nodo y si lo encuentra lo elimina, sino avisa que el nodo no se encuentra.
15. *Eliminar_Todo*: Borra nodo por nodo, y confirma si la eliminación se realizó con éxito o si tuvo algún problema.
16. *Lista*: Inicializa y prepara la creación de la lista.
17. *~Lista*: Reutiliza la función de eliminar la lista y borra el espacio de memoria.

Y, por último, el comportamiento de *Nodo* es la siguiente:

1. *Nodo*: Crea el nodo e inicializa todo a 0 o a nulo, correspondientemente.
2. *Nodo(int info)*: Sobrecarga del constructor, asigna el dato al nodo pero su puntero lo mantiene en nulo.
3. *Nodo(int info, Nodo* ptr)*: Sobrecarga del constructor, asigna el dato y la dirección a nodo.
4. *~Nodo*: Elimina el espacio de memoria ocupado por *Nodo*.

Conclusión

Una TDA es una manera bastante brillante de aplicar las herramientas que brinda la Programación Orientada a Objetos, las listas simplemente ligadas son una prueba de ello. Un beneficio de estas es que pueden implementarse sin importar el contexto y demás, acceder a todas sus funciones correctamente. Las operaciones implementadas en este caso no son todas las que se pueden realizar con las listas, pero, son las más esenciales para trabajar con ellas, que como principiante en la estructura de datos, fue un reto diseñar y programar cada una de ellas conociendo únicamente el concepto.